**RÉPUBLIQUE GABONAISE**
Union – Travail – Justice

**Ministère de l'Enseignement Supérieur**

**Centre Universitaire de Koulamoutou (CUK)**

**DUT Informatique – 1ère Année**

# RAPPORT FINAL

**PROJET SYSKIT**

**Automatisation des tâches Linux avec Bash et GitHub**

Cours : **Programmation Shell Bash**
Professeur : **M. Rolly Marvin ONDO ABAGHE**
Année académique : **2025 – 2026**


Membres du groupe :

- **NDONG NZE Jean De La France**
- **ADEM'AKOUE Graslya Luckariv Ineijda**
- **MAGNOUNGOU TANTAL Eveil Chris Claudel**

# Introduction

Dans le cadre du cours de programmation Shell Bash, nous avons été amenés à réaliser un projet en groupe consistant à développer un outil d'administration système sous Linux en utilisant le langage Bash et la plateforme GitHub. Ce projet, intitulé **SysKit**, avait pour objectif d'automatiser certaines tâches courantes que réalise un administrateur système, comme l'affichage des informations du système, la sauvegarde de dossiers, le nettoyage de fichiers inutiles et la génération de rapports.

L'objectif pédagogique de ce projet était de nous permettre de mettre en pratique les notions vues en cours, notamment l'utilisation des variables, des structures conditionnelles, des boucles, des fonctions, ainsi que la manipulation de fichiers en Bash. Ce projet nous a également permis d'apprendre à utiliser Git et GitHub pour le travail collaboratif, en suivant les modifications du projet à travers les commits et en travaillant à plusieurs sur un même dépôt.

Ce projet nous a aussi appris à organiser un projet informatique en respectant une structure de dossiers, à écrire des scripts clairs et commentés, et à travailler en équipe en se répartissant les tâches. Dans ce rapport, nous allons présenter le fonctionnement du programme SysKit, la structure du projet, l'explication des différents scripts, les captures d'écran des différentes étapes du projet, ainsi que la contribution de chaque membre du groupe.

## I.Description du programme SysKit

SysKit est un programme écrit en Bash qui permet d'automatiser certaines tâches d'administration sur un système Linux. Le programme principal est un script nommé **syskit.sh**, qui permet d'exécuter différentes fonctionnalités en fonction de l'argument saisi par l'utilisateur.

Le programme est composé de quatre fonctionnalités principales :

1. **Inventory** : permet d'afficher les informations du système.
2. **Backup** : permet de sauvegarder un dossier sous forme d'archive.
3. **Cleanup** : permet de supprimer les fichiers inutiles.
4. **Report** : permet de générer un rapport contenant plusieurs informations.

L'utilisateur exécute le programme à partir du terminal en utilisant les commandes suivantes :

```
./syskit.sh inventory
./syskit.sh backup /chemin/dossier
./syskit.sh cleanup /chemin
./syskit.sh report
```

Le script principal **syskit.sh** appelle les scripts situés dans le dossier **modules/** pour exécuter chaque fonctionnalité.

---

## II. Structure du projet

Pour une bonne organisation, le projet a été structuré en plusieurs dossiers et fichiers comme demandé dans le sujet.

Structure du projet :

```
syskit/
│
├── README.md
├── syskit.sh
├── modules/
│   ├── inventory.sh
│   ├── backup.sh
│   ├── cleanup.sh
│   └── report.sh
│
├── logs/
├── backup/
├── reports/
│
└── docs/
    └── rapport_final.pdf
```

### Explication de la structure

- **README.md** : contient les instructions d'utilisation.

- **syskit.sh** : script principal qui permet d'exécuter les différentes fonctionnalités.
- **modules/** : contient les scripts correspondant à chaque fonctionnalité.
- **logs/** : contient les fichiers de logs générés par le programme.
- **backup/** : contient les archives de sauvegarde.
- **reports/** : contient les rapports générés par le script report.
- **docs/** : contient le rapport final du projet.

Cette structure permet d'avoir un projet bien organisé et facile à comprendre.

---

## III. Explication des scripts

### 5.1 Script syskit.sh

Le fichier **syskit.sh** est le script principal du programme. Il permet de récupérer l'argument saisi par l'utilisateur et d'appeler le script correspondant.

Par exemple :

- si l'utilisateur tape `inventory`, le script appelle **inventory.sh**
- si l'utilisateur tape `backup`, le script appelle **backup.sh**
- si l'utilisateur tape `cleanup`, le script appelle **cleanup.sh**
- si l'utilisateur tape `report`, le script appelle **report.sh**

Ce script utilise des conditions `if` pour vérifier l'argument saisi.

---

### a. Script inventory.sh

Le script **inventory.sh** permet d'afficher les informations du système, notamment :

- le système d'exploitation
- la version du noyau Linux
- le processeur
- la mémoire RAM
- l'espace disque
- l'adresse IP

Ce script utilise plusieurs commandes Linux comme :

- `uname`
- `free`
- `df`
- `hostname`
- `ip`

Ces informations permettent de connaître l'état du système.

---

### b. Script backup.sh

Le script **backup.sh** permet de sauvegarder un dossier en créant une archive au format **.tar** dans le dossier **backup/**.

Le script :

- vérifie si le dossier à sauvegarder existe,
- crée le dossier **backup/** s'il n'existe pas,
- crée une archive avec la date dans le nom du fichier.

Exemple :

```
backup/documents_2026-03-05.tar
```

Cela permet de sauvegarder facilement des dossiers importants.

---

### c. Script cleanup.sh

Le script **cleanup.sh** permet de supprimer les fichiers inutiles comme :

- les fichiers `.tmp`
- les anciens fichiers `.log`

Le script parcourt les fichiers avec une boucle `for` et supprime les fichiers correspondants avec la commande `rm`.
Cela permet de libérer de l'espace disque et de garder un système propre.

---

### d. Script report.sh

Le script **report.sh** permet de générer un fichier **report.txt** contenant :

- les informations système,
- la liste des sauvegardes réalisées,
- le nombre de fichiers supprimés.

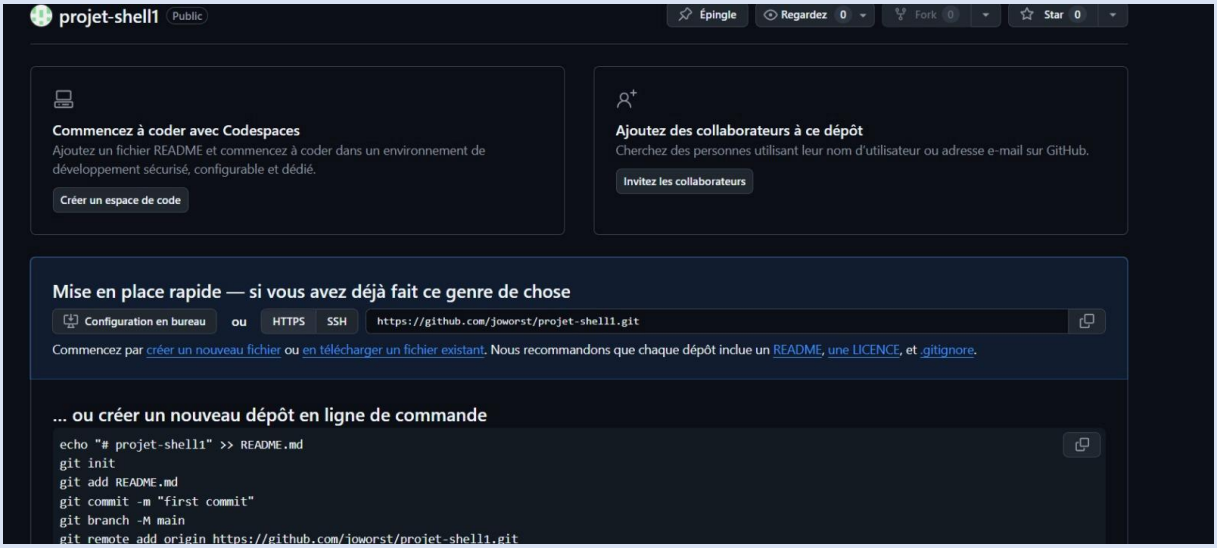
Le rapport est enregistré dans le dossier **reports/**.
Ce script permet d'avoir un résumé de l'activité du programme SysKit.

---

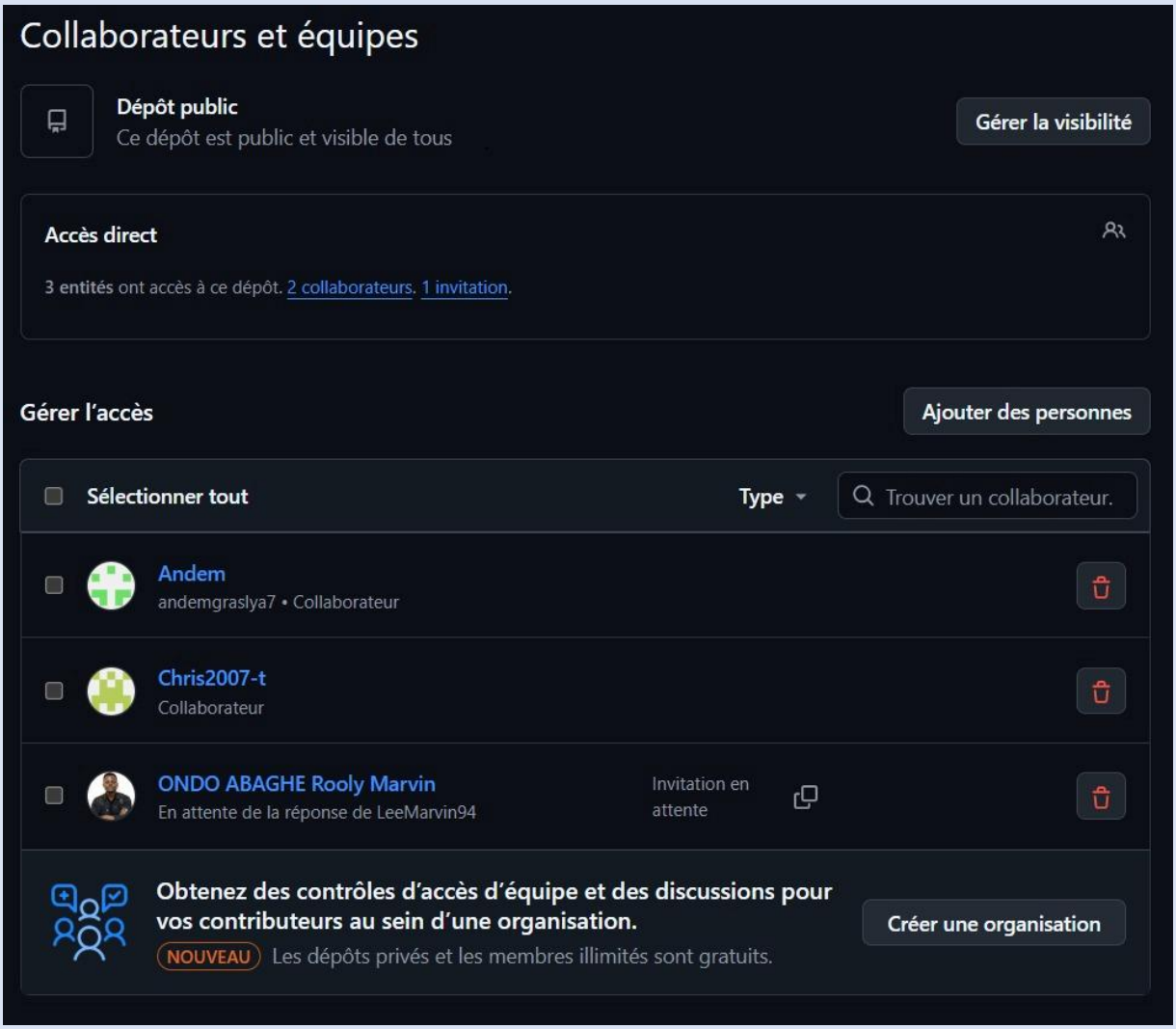
## IV. Captures d’écran

Dans cette partie, nous avons ajouté les captures d’écran suivantes :

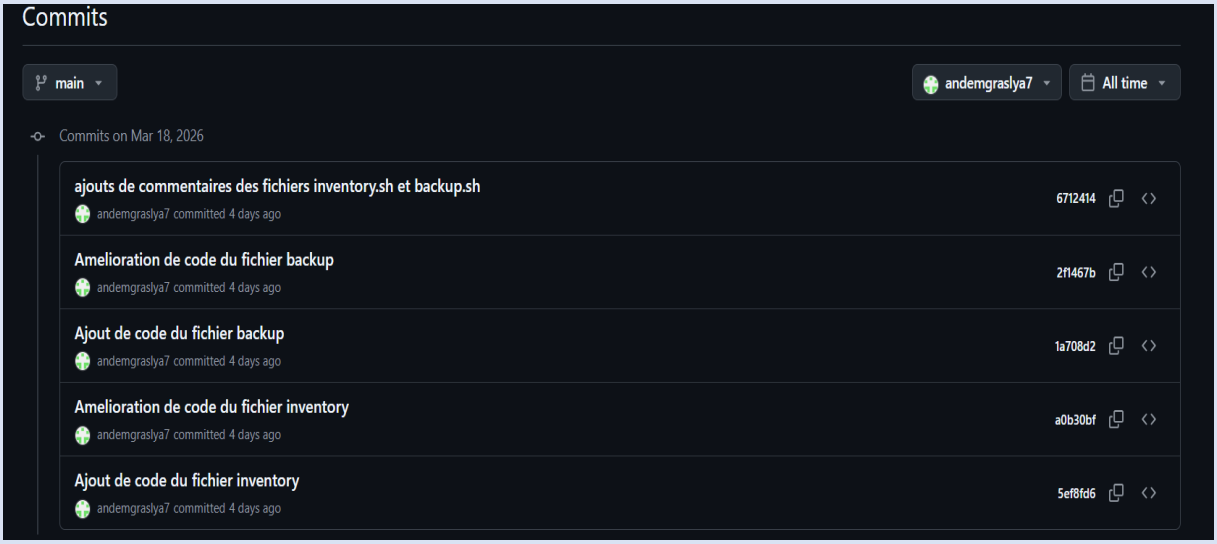
1. Création du repository GitHub

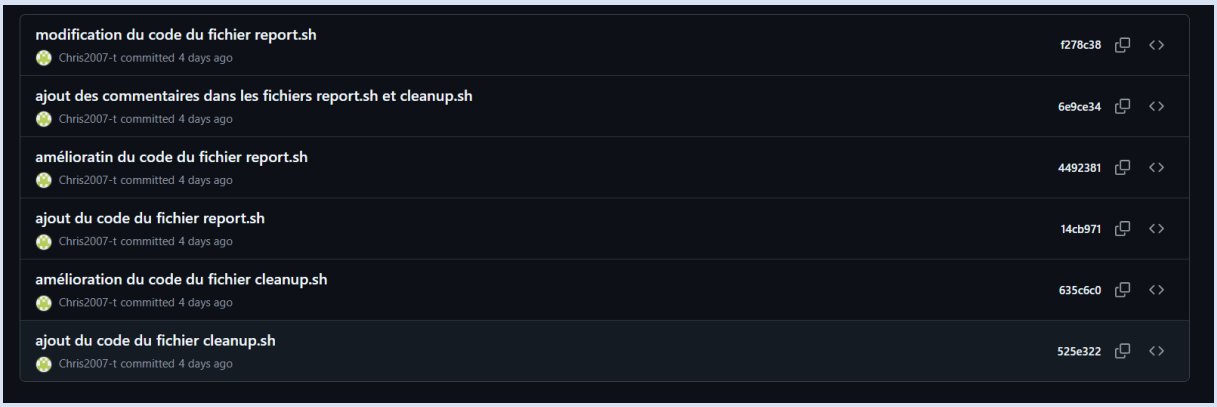


2. Ajout des collaborateurs (membres du groupe et professeur)




3. Historique des commits






4. Exécution de la commande inventory

```
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ ./syskit.sh inventory
===== INVENTORY =====
Système d'exploitation :
GNU/Linux
Version du noyau :
6.6.87.2-microsoft-standard-WSL2
Processeur :
Model name:                          13th Gen Intel(R) Core(TM) i5-13420H
Mémoire RAM :
              total        used        free      shared  buff/cache   available
Mem:          3.7Gi       463Mi       2.6Gi       3.8Mi       678Mi       3.2Gi
Swap:         1.0Gi          0B       1.0Gi
Espace disque :
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G     0  1.9G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            1.9G  4.0K  1.9G   1% /mnt/wsl
drivers         477G  188G  289G  40% /usr/lib/wsl/drivers
/dev/sdd       1007G  2.7G  954G   1% /
none            1.9G   72K  1.9G   1% /mnt/wslg
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  2.7M  1.9G   1% /init
none            1.9G  832K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G     0  1.9G   0% /run/shm
none            1.9G   76K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   76K  1.9G   1% /mnt/wslg/doc
C:\             477G  188G  289G  40% /mnt/c
snapfuse         74M   74M     0 100% /snap/core22/2339
snapfuse         74M   74M     0 100% /snap/core22/2216
snapfuse         49M   49M     0 100% /snap/snapd/26382
snapfuse         49M   49M     0 100% /snap/snapd/25935
snapfuse        4.5M  4.5M     0 100% /snap/tree/54
tmpfs           378M   20K  378M   1% /run/user/1000
Adresse IP :
172.21.244.132
```

5. Exécution de la commande backup

```
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ ./syskit.sh backup modules/
modules/
modules/cleanup.sh
modules/inventory.sh
modules/report.sh
modules/backup.sh
Sauvegarde créée : backup/modules_2026-03-22.tar
```

6. Exécution de la commande cleanup

```
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ touch docs/love.log
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ ./syskit.sh cleanup docs/
Supprimé : docs//love.log
1 fichiers supprimés
```

7. Génération du rapport avec report

```
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ ./syskit.sh report

Rapport généré dans reports/report.txt
ideal@DESKTOP-F1B3CHG:~/projet-shell1/syskit$ cat reports/report.txt
===== RAPPORT SYSKIT =====
Date : Sun Mar 22 14:04:40 WAT 2026

===== INFORMATIONS SYSTEME =====
Linux DESKTOP-F1B3CHG 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux

===== SAUVEGARDES REALISEES =====
backup/modules_2026-03-22.tar

===== NETTOYAGE =====
Nombre total de fichiers supprimés : 1
```

Chaque capture d'écran montre le bon fonctionnement du programme.

## V. Contribution des membres

Chaque membre du groupe devait réaliser au moins 5 commits sur GitHub. Le tableau suivant présente la contribution de chaque membre.

### Tableau – Contribution des membres

| Nom | Nombre de commits | Travail réalisé |
|---|---|---|
| NDONG NZE Jean De La France | 6 commits | NDONG-NZE à realiser 6commit: 1er commit : structure du projet ; 2ème commit : ajouts des permissions d'exécution des scripts ; 3ème commit : ajout de code du script syskit.sh ; 4ème commit : ajout de textes du fichier README.md ; 5ème commit : ajout des commentaires ; 6ème commit : ajout du rapport final dans le dossier docs. |
| ADEM'AKOUE Graslya Luckariv Ineijda | 5 commits | ANDEM'AKOUE a fait 5 commits répartis sur deux scripts (inventory et backup). Commit 1 : ajout de code du fichier inventory ; Commit 2 : amélioration de code du fichier inventory ; Commit 3 : ajout de code du fichier backup ; Commit 4 : amélioration de code du fichier backup ; Commit 5 : ajouts de commentaires des fichiers inventory.sh et backup.sh. |
| MAGNOUNGOU TANTAL Eveil Chris | 5 commits | MAGNOUNGOU a réalisé aussi 6 commits. Commit 1 : ajout de code du fichier cleanup.sh ; Commit 2 : amélioration de code du fichier cleanup.sh ; Commit 3 : ajout de code du fichier report.sh ; Commit 4 |

| Nom | Nombre de commits | Travail réalisé |
|---|---|---|
| Claudel | | : amélioration de code du fichier report.sh ; Commit 5 : ajouts de commentaires des fichiers cleanup.sh et report.sh. Chris a aussi apporté son aide dans les configurations et les commandes GitHub. |

## Autres contributions de chaque membre.

**NDONG-NZE :**

Avant de faire ses commits, Jean a tout d'abord créé le repository, créé la structure du projet, créé syskit.sh, ajouté le README et corrigé le projet. Ensuite, il a aidé les membres de son groupe à comprendre l'ensemble du projet et à régler les différents problèmes rencontrés au cours de leur travail. Puis il a réalisé 6 commits

**ANDEM'AKOUE :**

Apres la realisation de ses 5 commits ANDEM' a realisée le rapport final du projet

MAGNOUNGOU TANTAL **:**

Pendant la realisation du projet Chris a aidé pour les configurations et les commandes GitHub puis a realisé ses commits.

## Conclusion

Ce projet nous a permis de mettre en pratique les connaissances apprises en programmation Shell Bash et d'apprendre à travailler en équipe avec Git et GitHub. Nous avons appris à structurer un projet informatique, à écrire des scripts Bash, à automatiser des tâches sous Linux et à utiliser un système de gestion de versions.

Nous avons rencontré quelques difficultés, notamment dans la création de certains scripts, la gestion des erreurs, et l'utilisation de GitHub, mais nous avons réussi à résoudre ces problèmes en travaillant en groupe et en nous aidant mutuellement.

Ce projet a été très enrichissant car il nous a permis de mieux comprendre le rôle de la programmation Shell dans l'administration système et l'importance du travail collaboratif dans un projet informatique.

---